

이미지·센서 기반 빗물받이 위험도 모니터링

우수주의보

이미지·센서 기반 빗물받이 위험도 판단 및 관리자
모니터링 시스템

상태 모니터링

지도 기반 시설 확인

위험도 분류

이미지·센서 통합 판단

점검 우선순위

위험 시설 우선 확인

Team 우수주의보 · 최종 프로젝트 발표 · 2026.07.02

☰  지능형 도시 침수 관리 대시보드

🏠 🔔 👤

대시보드 현황
API 응답 기준 · 최신 업데이트 2026-06-23 23:35:09

전체
5

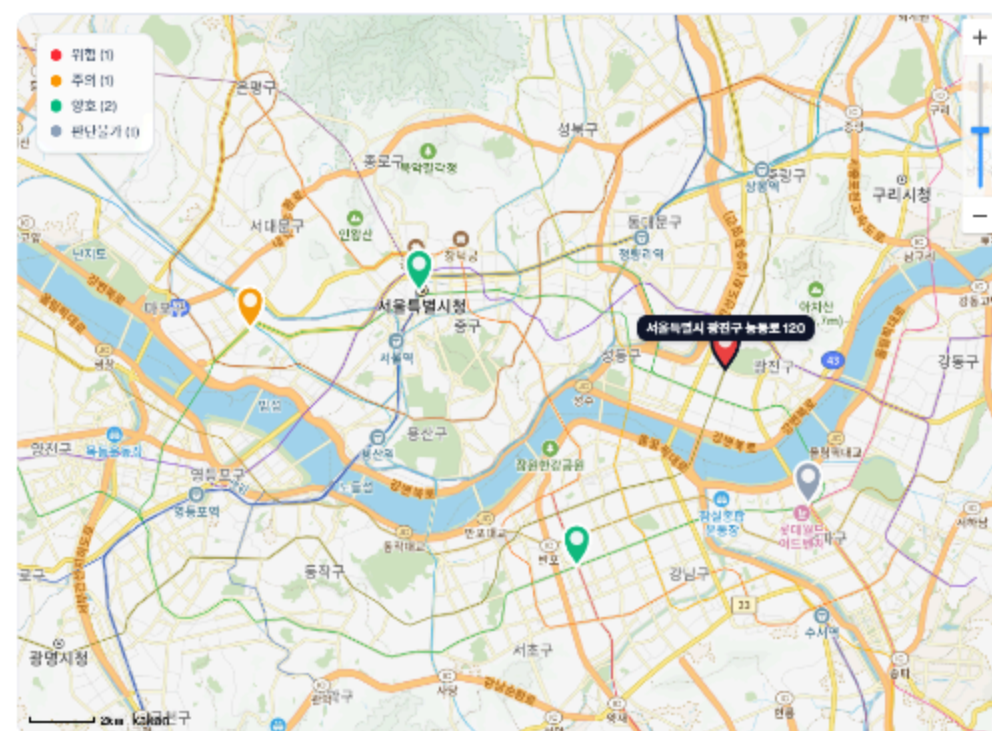
위험
1

주의
1

양호
2

판단불가
1

도시 배수 시설 위험도 지도



① 지도에서 배수 시설은 클릭하면 상세 정보를 확인할 수 있습니다.

위험 시설 목록

● 실시간 반영됨

위험도순 ▼

1 **DR-004** 서울특별시 광진구 능동로 120 **위험**
현황 결과 **악천후**가 수위가 높아 침수 위험이 큼니다.
악천후 정도 88% 수위 85 cm
최근 업데이트 2026-06-23 23:30:09

2 **DR-003** 서울특별시 마포구 양화로 160 **주의**
현황 결과 **일부 악천후**가 감지되어 주의 관찰이 필요합니다.
악천후 정도 62% 수위 48 cm
최근 업데이트 2026-06-23 23:25:09

3 **DR-005** 서울특별시 송파구 올림픽로 265 **판단불가**
현황 결과 센서 및 이미지 신뢰도가 낮아 위험도를 판단하기 어렵습니다.
악천후 정도 0% 수위 0 cm
최근 업데이트 2026-06-23 23:35:09

4 **DR-002** 서울특별시 강남구 강남대로 396 **양호**

상세 정보

서울특별시 광진구 능동로 120
(빗물받이)

상태 **위험**
2026-06-23 23:30:09
현황 결과 **악천후**가 수위가 높아 침수 위험이 큼니다.

악천후 정도 88%
수위 85 cm
유속 0.05 m/s



주소 서울특별시 광진구 능동로 120
최근 업데이트 2026-06-23 23:30:09

[상세 정보 페이지로 이동 >](#)

목차

문제 정의부터 시연 흐름까지 순서대로 설명합니다.

01

개발 필요성과 벤치마킹

왜 이 서비스가 필요한가?

02

AI 분석 설계

이미지를 어떻게 판단 근거로
바꾸는가?

03

웹 서버 및 서비스 구축

결과는 어떻게 저장되고 화면에
반영되는가?

04

DB / 아키텍처 / 통신 설계

데이터와 화면 갱신 흐름은 어떻게
연결되는가?

05

시연과 마무리

어떤 흐름으로 동작을 보여주는가?

프로젝트 한 줄 요약

빗물받이의 이미지와 센서값을 분석해
위험 시설을 먼저 확인할 수 있게 합니다.

01

입력

시연용 이미지와 센서값을 입력합니다.

02

분석

막힘 정도, 수위, 유속을 함께 판단합니다.

03

저장

분석 결과와 상태 이력을 데이터베이스에
저장합니다.

04

반영

관리자 화면에서 위험 시설 상태를
확인합니다.

팀 구성과 역할

송희수

PM / AI 분석

- YOLO / OpenCV
- 이미지 라벨링
- AI 파이프라인 관리

이명근

Backend / DB

- FastAPI
- PostgreSQL
- callback / WebSocket

오택률

Frontend / Infra / 발표

- 관리자 대시보드 구현
- 시설 상세 화면 구현
- Docker / Jenkins 배포 구성

김운섭

XGBoost / AI 연동

- 위험도 분류 기준 설계
- XGBoost 입력 feature 정리
- AI 서버 ↔ Backend 통신 흐름 정리

API 명세 공유 · 단위 통일 · 통합 점검 · 시연 정리

MVP 기술 스택 요약

**MVP는 화면, API, AI
분석, 저장소,
배포와 협업 도구를
역할별로 분리해
구성했습니다.**

각 영역의 책임을 나눠 개발·점검·배포 흐름을
관리했습니다.

Next.js / React

FastAPI

YOLO / OpenCV / XGBoost

PostgreSQL

Docker / Nginx / Jenkins

Slack / Discord / FigJam

프론트엔드

Next.js / React
관리자 대시보드
WebSocket 화면 반영

백엔드

FastAPI
Scheduler / REST API
Callback 수신·상태 관리

AI 서버

YOLO 객체 탐지
OpenCV 후처리
XGBoost 위험도 분류

데이터베이스

PostgreSQL
sensor_data 저장
AI 분석 결과 저장

인프라 / 배포

Docker Compose
Nginx 라우팅
Jenkins 배포 자동화

협업 도구

Slack
Discord
FigJam

도시 침수와 빗물받이

빗물받이 하나가 막히면 해당 지점의 배수 흐름이 제한됩니다.

목표는 도시 전체 침수 예측이 아니라, 개별 빗물받이의 상태 변화를 빠르게 확인하는 것입니다.

- 빗물받이는 도로 빗물이 배수관으로 들어가는 첫 지점입니다.
- 낙엽, 쓰레기, 토사로 막히면 강우 중 배수 흐름이 제한됩니다.
- 개별 시설 상태를 확인해야 점검 우선순위를 잡을 수 있습니다.



이물질로 막힌 빗물받이 예시

강우 중 상태 변화

강우 중에는 상태가 빠르게 변하므로,
변화가 화면에 반영되는 흐름이
필요합니다.

01

강우

빗물이 유입됩니다.

02

유입량 증가

시설별 유량 차이가 커집니다.

03

차폐 증가

이물질이 유입부를 가립니다.

04

수위 상승

센서값에서 수위 변화를 확인합니다.

05

유속 저하

흐름 둔화가 판단 근거가 됩니다.

06

상태 갱신

화면이 최신 상태를 반영합니다.

여러 시설의 상태 변화가 같은 시간대에 발생할 수 있어 화면 기반 확인 흐름이 필요합니다.

기존 관리 방식에서 확인한 한계

기존 방식은 위치 확인, 이력 관리, 현장 접수에는 강점이 있지만, 강우 중 빠르게 변하는 시설 상태를 선제적으로 판단하기에는 한계가 있었습니다.

방식	강점	한계
정기 점검	기준과 이력 관리 가능	강우 중 급격한 상태 변화를 즉시 반영하기 어려움
시민 신고	현장 발견과 접수에 강점	신고가 들어오기 전까지 선제 대응이 어려움
QR·관리번호	위치 특정과 이력 연결에 유리	시설 추적은 가능하지만 상태 판단은 별도로 필요함
센서·관제	수치 기반 상태 확인 가능	설치·유지관리 비용과 운영 부담이 큼

결론: 시설별 상태를 자동으로 분류하고, 그 결과를 화면에 빠르게 반영하는 흐름이 필요합니다.

사후 확인에서 우선 확인으로

신고, 점검 뒤에 확인하던
방식을, 시설별 상태를
먼저 확인하는 방식으로
바꿉니다.

시민 신고를 대체하기보다, 이미지·센서 근거를
저장하고 화면에 반영해 점검 우선순위 판단을
보조합니다.

상태 분류

이미지 근거

센서 근거

상세 이력

화면 반영

구분	기존 방식	우수주의보
문제 확인	점검·신고 이후 파악	시설별 상태를 먼저 분류
판단 근거	현장 확인 중심	이미지 feature + 센서 feature
관리 단위	시설 이력 중심	개별 빗물받이 + 분석 작업
결과 활용	기록과 사후 대응	우선 확인 대상과 상세 근거 제공

정책·사례에서 확인한 운영 목표



그림 8. 빗물받이 관리를 위한 DB구축 및 스마트 유지관리 방안

시설 데이터 통합

빗물받이별 센서값, 이미지 분석 결과, 상태 이력을 같은 시설 기준으로 묶습니다.

상태 변화 확인

강우 중 변하는 현장 상태를 분석 결과와 함께 화면에서 확인할 수 있게 합니다.

점검 우선순위 지원

관리자가 먼저 확인해야 할 시설을 고를 수 있도록 상태 분류와 근거를 제공합니다.

출처: 물과미래 Vol.58 No.6, 「배수기능 확보를 위한 빗물받이 설치 및 유지관리 기준 마련」

프로젝트 진행 일정

6/15-18	6/19-23	6/24-28	6/29-7/1	7/2
기획·설계	MVP 개발	통합·개선	검증·리허설	최종 발표
요구사항 정리 MVP 범위 설정 와이어프레임 ERD 설계	대시보드 API DB YOLO·XGBoost	WebSocket UI 개선 시연 데이터	통합 점검 자료 정리 리허설	시연 질의응답

AI 분석 설계 개요

이미지 분석 결과와 센서값을 결합해 최종 상태 분류로 연결합니다.

01

원본 이미지

sample image를 분석 입력으로 받습니다.

02

YOLO

보이는 객체와 후보 영역을 탐지합니다.

03

OpenCV

탐지 결과와 이미지 영역을 막힘 비율로
수치화합니다.

04

XGBoost

이미지 feature와 센서 feature를 함께
사용합니다.

05

상태 분류

양호, 주의, 위험, 판단불가로 저장합니다.

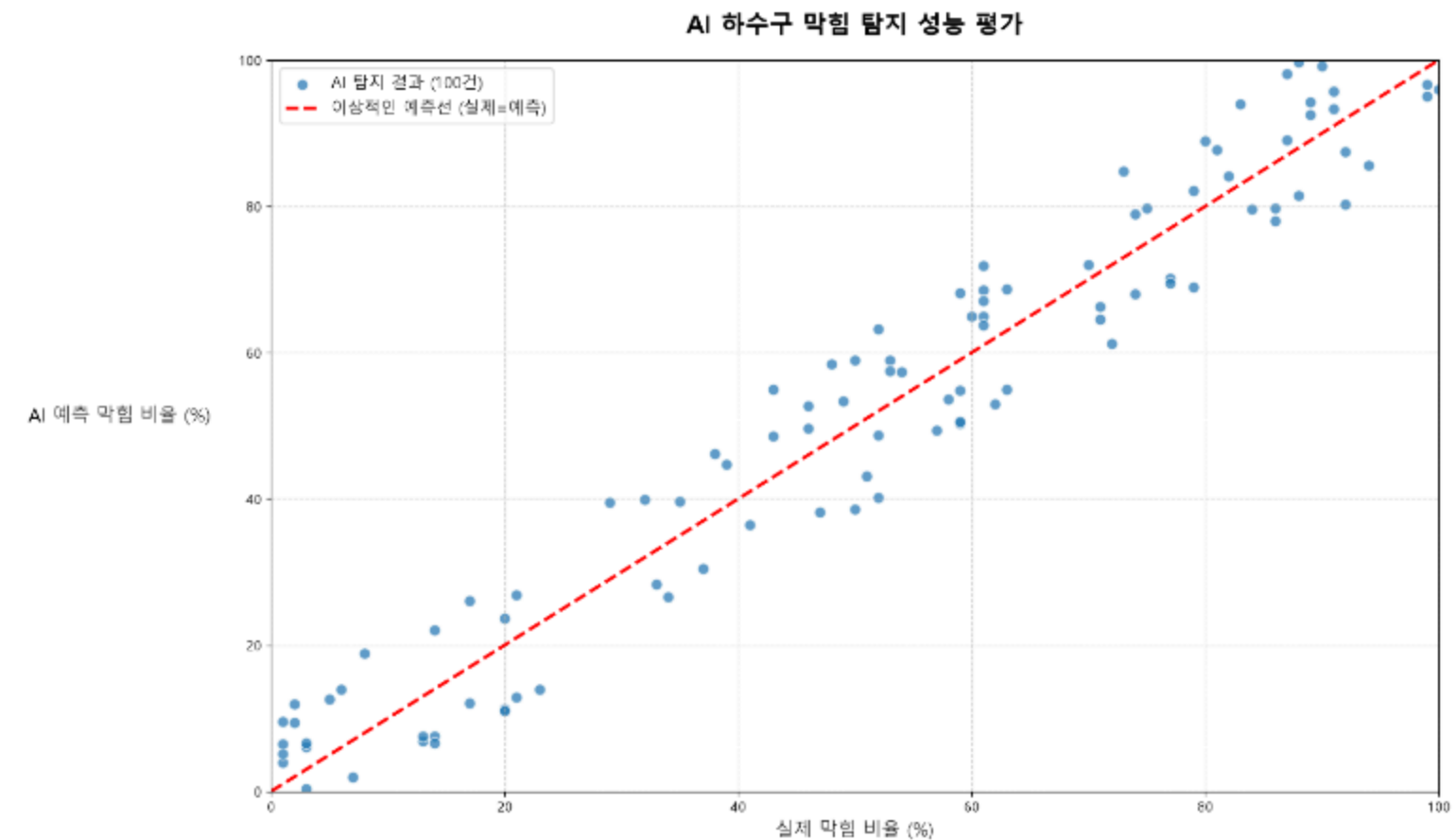
YOLO는 후보 영역을 찾고, OpenCV는 막힘 비율을 수치화하며, XGBoost는 이미지·센서 feature를 결합해 상태를 분류합니다.

YOLO 단독 분석 한계

객체 탐지만으로는 배수 위험도를 안정적으로 판단하기 어렵습니다.

탐지 결과를 그대로 위험도로 쓰지 않고, OpenCV 후처리와 센서 feature를 결합해 상태 분류로 연결합니다.

- YOLO는 보이는 객체와 후보 영역 탐지에 강점이 있습니다.
- 어두운 이미지, 빗물 반사, 낙엽/토사 혼재 상황은 해석을 흔들 수 있습니다.
- 그래서 OpenCV 후처리와 센서 feature를 함께 사용합니다.



개념 예시: 객체 탐지만으로 상태 분류가 흔들릴 수 있어 OpenCV·센서 feature를 함께 사용합니다.

OpenCV 결합 이유

AI PIPELINE

OpenCV는 탐지 결과를 분류에 사용할 feature로 정리하는 중간 단계입니다.

분석 영역과 차폐 비율을 정리해 XGBoost 입력으로 연결합니다.

원본 이미지

sample image를 분석 입력으로 사용합니다.



OpenCV 전처리

대비와 경계를 보정합니다.



mask / dark area

어두운 영역과 mask를 분리합니다.



ratio

차폐 영역을 obstruction_ratio로 정리합니다.



XGBoost 입력 feature

XGBoost에는 이미지 feature와 센서 feature가 함께 들어가며, 근거가 부족하면 판단불가 또는 보수적 분류 후보가 됩니다.

feature	출처	의미	결측 처리 기준
obstruction_ratio	OpenCV 후처리	이미지 기반 막힘 정도	품질 낮으면 판단불가 후보
confidence_score	YOLO 결과	탐지 신뢰도	낮으면 이미지 근거 제한
water_level	mock sensor data	수위 변화	없으면 센서 근거 부족
flow_velocity	mock sensor data	유속 변화	없으면 이미지 근거 중심

핵심은 이미지 근거와 센서 근거를 분리해 저장하고, 최종 분류에서는 함께 조합한다는 점입니다.

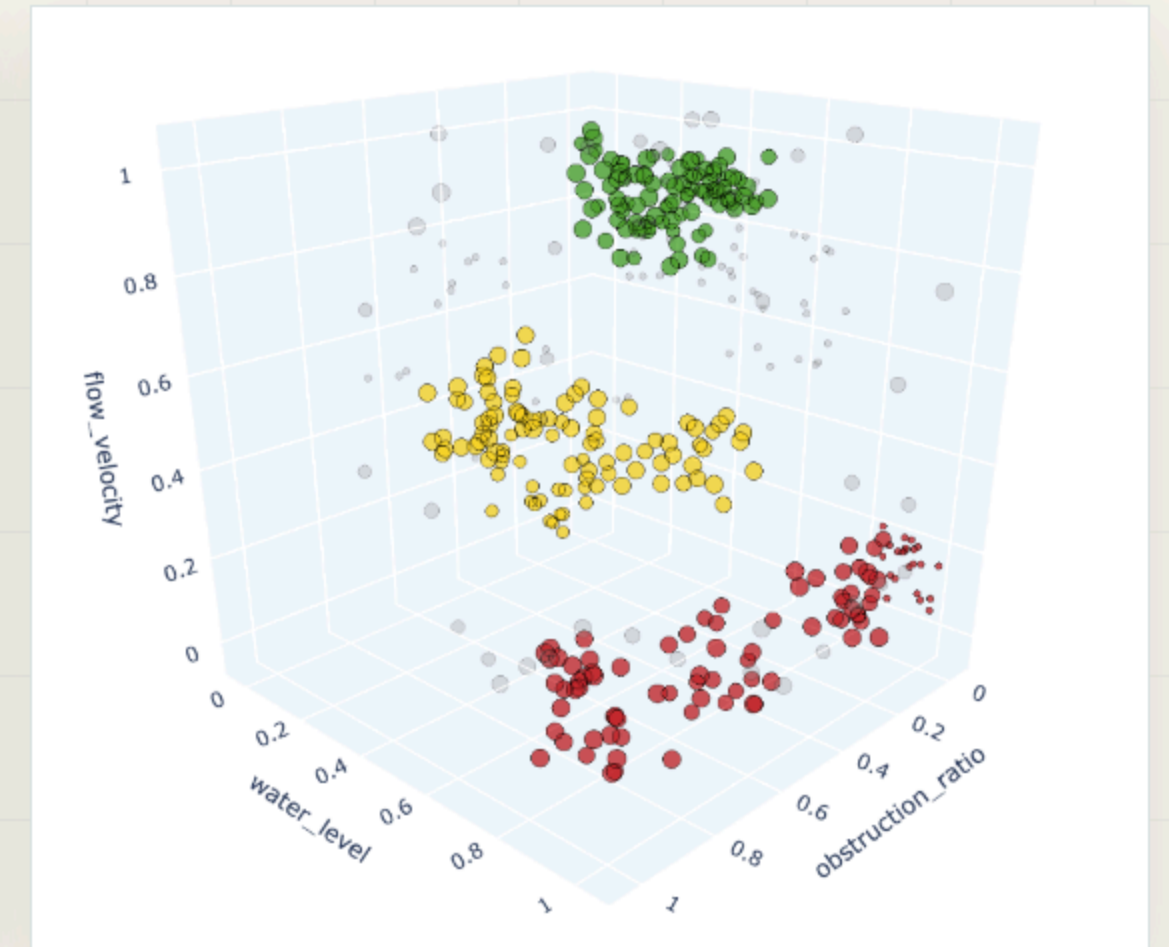
sensor_data 원본 단위는 water_level_cm, flow_velocity_mps이며, AI service 내부에서 정규화해 사용합니다.

위험도 분류 로직

상태 분류는 점검 우선순위를 설명하기 위한 내부 계산 결과이며, 실제 침수 확률을 직접 의미하지 않습니다.

아래 조건은 발표 설명용 대표 예시이며, 실제 결과는 이미지 feature와 센서 feature를 함께 반영한 최종 분류 결과입니다.

상태	조건 예시	화면 처리	발표 표현
양호	차폐율과 센서 정체 징후가 낮음	일반 상태로 표시	우선 점검 필요 낮음
주의	막힘 또는 수위·유속 변화 징후가 있음	목록에서 점검 후보 표시	관찰 필요
위험	이미지와 센서 근거가 함께 강함	대시보드 우선 확인 대상으로 표시	우선 확인 필요
판단불가	이미지 품질 낮음, feature 결측, 분석 실패	재분석 또는 현장 확인 상태로 표시	재확인 필요



`risk\_score`는 우수주의보 내부 분류용 계산 점수이며 실제 현장 검증 기반 침수 예측값이 아닙니다.

AI 결과 활용 흐름

AI 분석 결과는 관리자 화면에서 시설 상태와 점검 우선순위를 확인하는 흐름으로 연결됩니다.

01

대상 확인

관리자가 대시보드에서 시설 목록과 상태를 확인합니다.

02

분석 요청

sample image와 mock sensor data를 분석 작업으로 묶습니다.

03

결과 저장

callback으로 받은 상태 분류 결과를 DB에 저장합니다.

04

화면 반영

WebSocket event를 통해 dashboard/detail 화면 상태를 갱신합니다.

현재 구현 범위는 sample/mock 기반 MVP이며, 실제 CCTV나 실센서 연동은 향후 확장 대상으로 분리합니다.

관리자 대시보드 흐름

상태 요약, 위험 시설 목록, 지도, 선택 시설 정보를 한 화면에서 확인합니다.

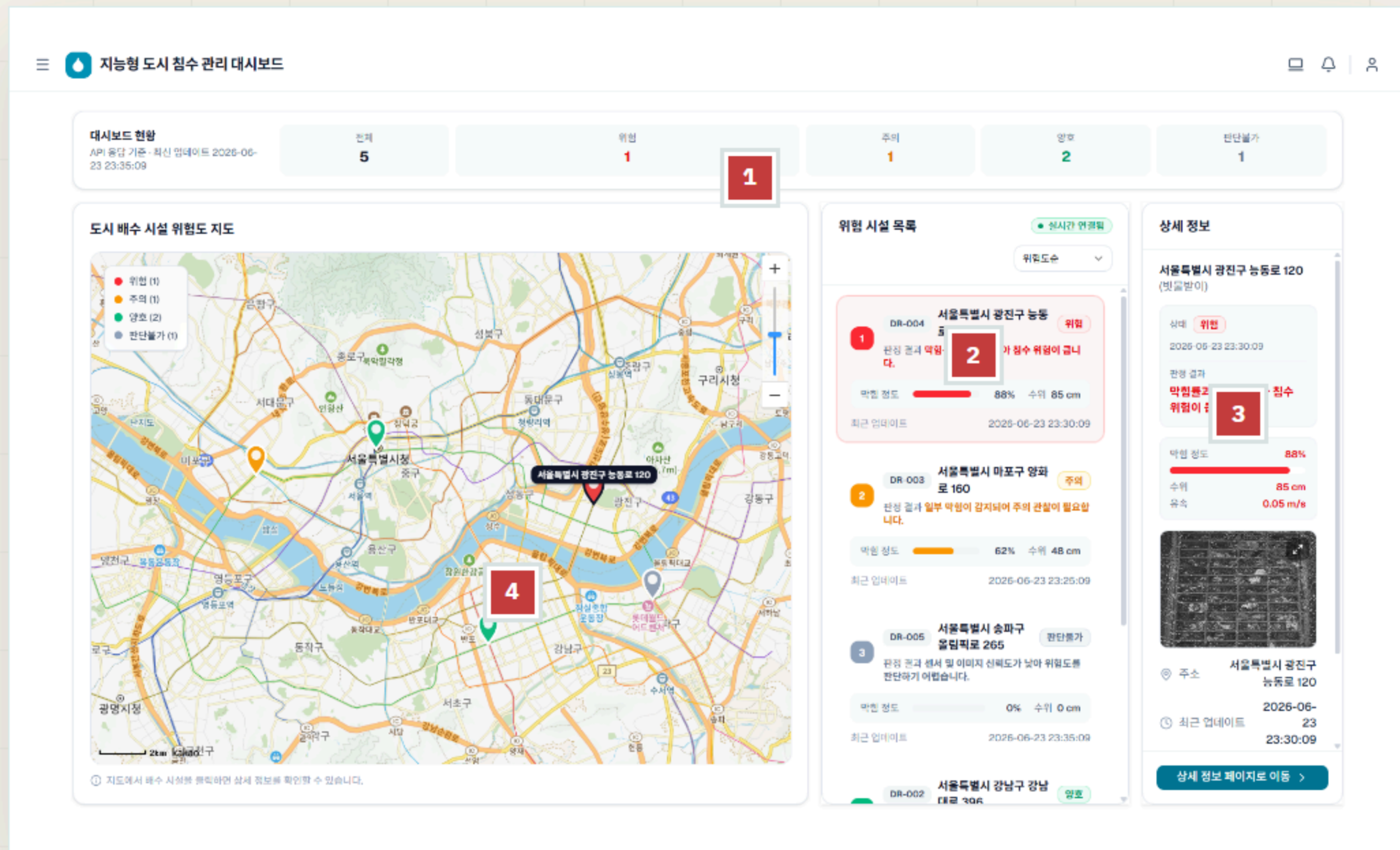
운영자는 먼저 확인할 시설을 지도와 목록에서 고르고, 선택 시설의 요약 정보를 함께 확인합니다.

1 전체 상태 요약

2 위험 시설 목록

3 선택 시설 요약

4 지도/마커 상태



관리자 대시보드: 지도, 위험 리스트, 선택 시설 요약, 상태 갱신을 함께 확인

상세 화면 흐름

한 시설의 현재 상태와 판단 근거를 동시에 봅니다.

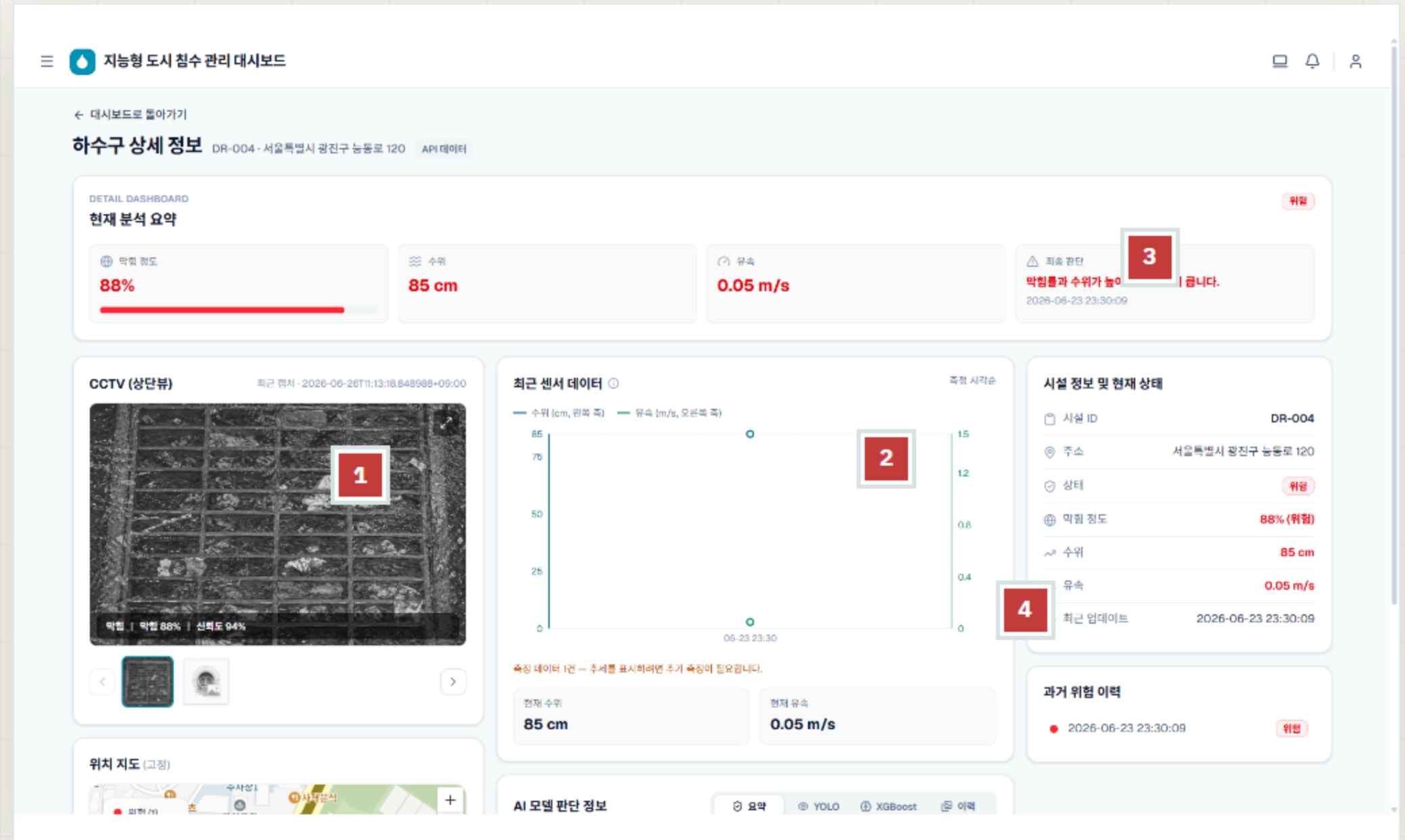
이미지 분석, 수위·유속 추세, 위험도 요약, 시설 메타정보를 한 화면에서 확인합니다.

1 이미지 분석 결과

2 센서 시계열

3 위험도 요약

4 시설 정보/상태 이력



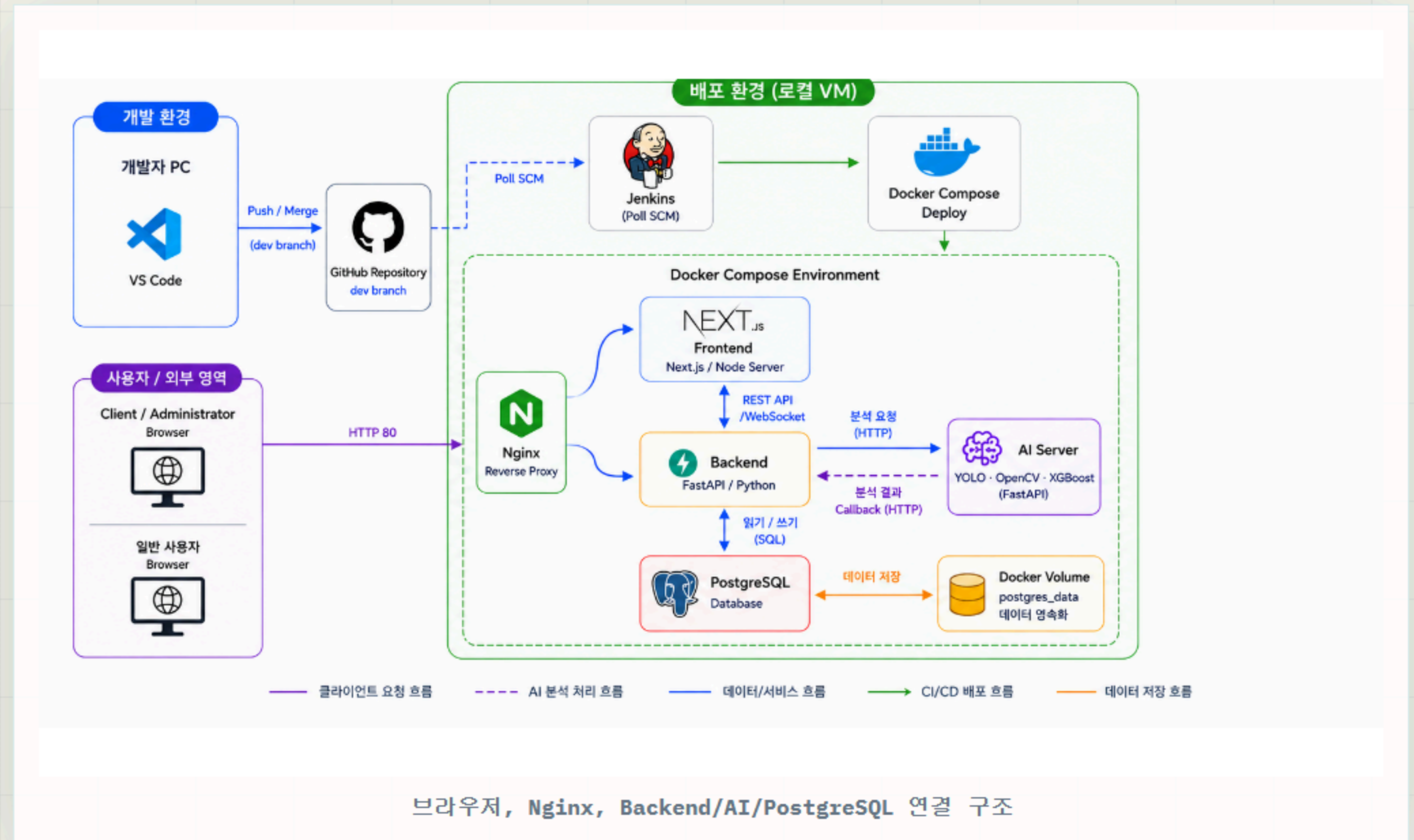
상세 화면: 이미지·센서·AI 결과와 저장된 이력을 시설 단위로 확인

시스템 아키텍처

배포 관점에서는 Nginx가
진입점이 되고, 각
서비스는 컨테이너 단위로
분리됩니다.

현재 구현과 향후 확장을 solid/dotted line으로
구분합니다.

☐ Nginx routing
 ☐ Frontend container
 ☐ Backend API
 ☐ AI service
 ☒ PostgreSQL volume



DB 설계 / ERD

센서·이미지·위험도를 같은 시설 기준으로 묶습니다.

시설 기준

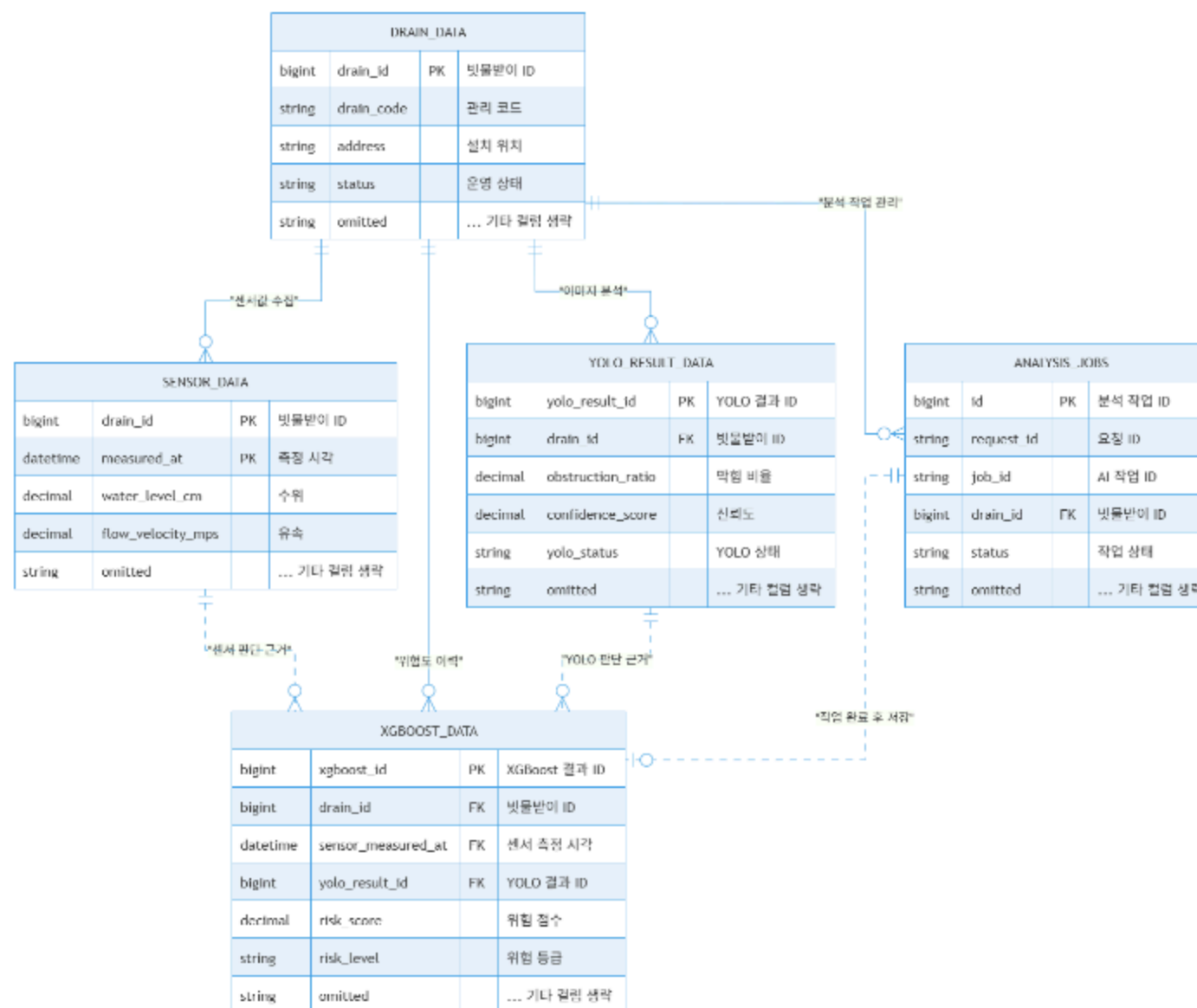
빗물받이 시설을 중심으로 센서값, 이미지 분석 결과, 최종 위험도 결과를 연결합니다.

판단 근거 분리

센서 측정값과 이미지 분석 결과를 분리해 저장하고, 최종 분류에서 함께 사용합니다.

작업 관리

분석 요청과 완료 상태는 별도 작업 단위로 관리합니다.



센서 데이터에 복합키를 적용한 이유

센서 데이터는 “어느 시설의 어느 시각 값인가”가 핵심이므로, `drain_id + measured_at` 조합으로 식별하는 것이 자연스럽습니다.

선택한 기준

`drain_id + measured_at`

같은 빗물받이에서도 시간대별 센서값이 계속 쌓이기 때문에, 시설 ID와 측정 시각을 함께 사용해 센서 row를 구분합니다.

Q1 최근 센서 조회

0.1842ms

복합키 유리

Q2 경고 대시보드

0.3342ms

복합키 유리

Q3 추적 조인

0.3071ms

단일 ID 유리

결론: 센서 시계열 데이터는 복합키를 사용하고, YOLO·XGBoost 결과처럼 추적 조인이 많은 데이터는 단일 ID로 관리합니다.

분석 요청부터 화면 갱신까지

전체 시퀀스를 먼저 보고, AI 분석 구간과 화면 갱신 구간을 나눠서 확인합니다.

백엔드는 분석 대상 확인, AI 요청, 결과 저장, 화면 갱신을 연결하는
중심 역할을 합니다.

01 전체 구조

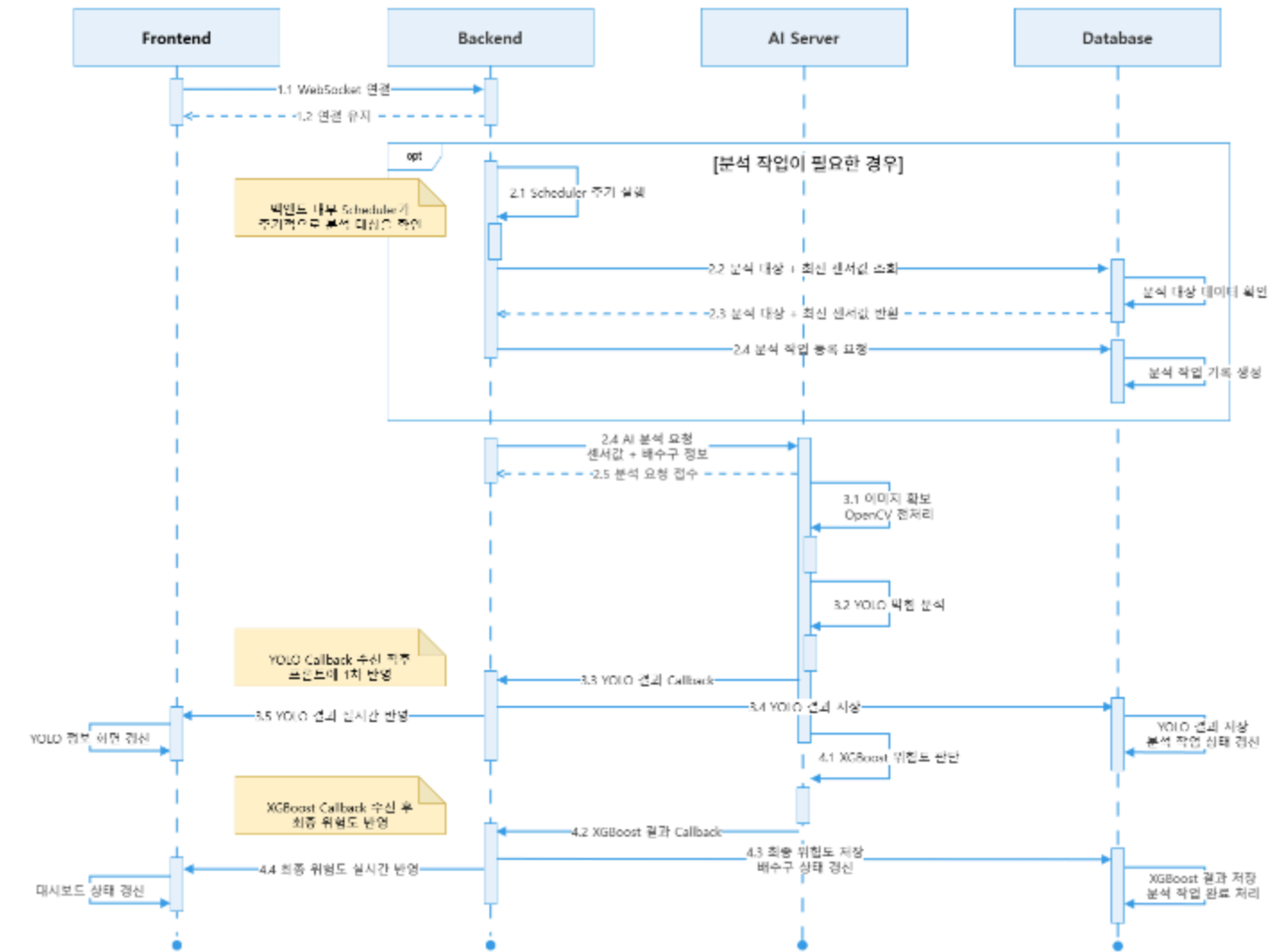
분석 대상 확인부터 화면 반영까지의 큰 순서를 먼저 잡습니다.

02 AI 분석 구간

백엔드가 입력을 준비하고, AI 서버가 결과를 콜백으로 반환합니다.

03 화면 갱신 구간

저장된 결과 변화는 WebSocket 이벤트로 화면에 반영됩니다.



전체 시퀀스는 구조 확인용이며, 다음 페이지에서 주요 구간을 나눠 설명합니다.

AI 분석 요청과 결과 반환

백엔드가 입력을 준비하고, AI 서버가 분석 결과를 반환합니다.

화면 응답과 AI 처리 시간을 분리하기 위해 분석은 작업 단위로 넘기고, 결과는 콜백으로 받아 저장합니다.

01 분석 대상 확인

Scheduler가 분석이 필요한 배수구를 확인합니다.

02 최신 센서값 조회

분석 시점에 가까운 수위와 유속 값을 가져옵니다.

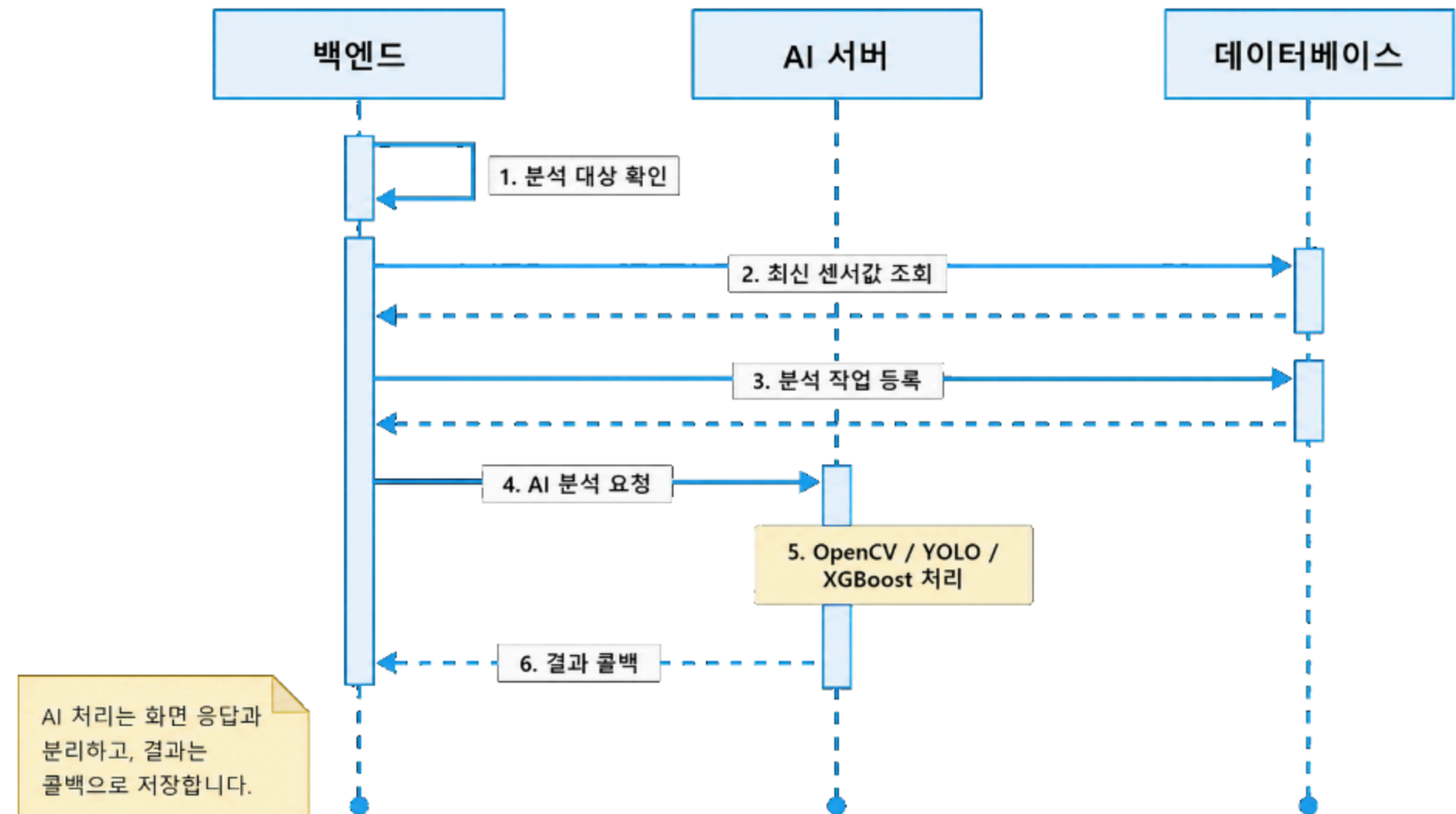
03 AI 분석 요청

배수구 정보와 센서값을 AI 서버 입력으로 전달합니다.

04 결과 콜백

OpenCV, YOLO, XGBoost 처리 결과를 백엔드로 반환합니다.

핵심은 AI 서버가 DB를 직접 수정하지 않고, 백엔드 콜백을 통해 결과를 저장한다는 점입니다.



AI 분석 구간: 대상 확인 → 센서값 조회 → 분석 요청 → 결과 콜백

WebSocket 기반 화면 갱신

저장된 결과 변화를 화면에 알리는 이벤트 채널입니다.

WebSocket은 분석 데이터 전체를 운반하는 통로가 아니라, 결과가 바뀌었음을 프론트엔드에 알려 화면 상태를 갱신하게 합니다.

01 연결 유지

Frontend와 Backend가 WebSocket 연결을 유지합니다.

02 YOLO 결과 반영

탐지 결과 저장 이후 상세 화면에 1차 결과를 반영합니다.

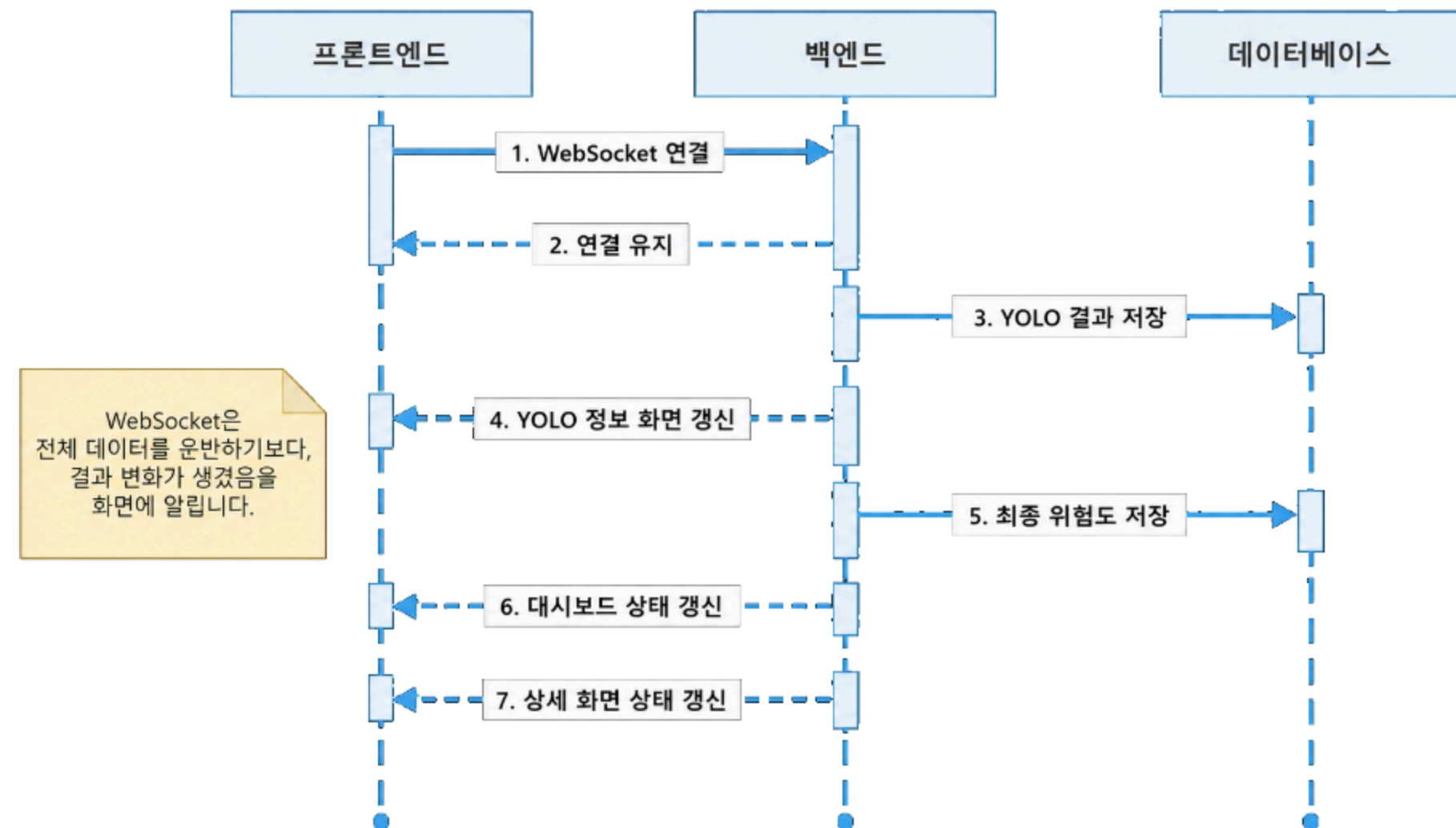
03 최종 위험도 반영

XGBoost 결과 저장 이후 배수구 상태를 갱신합니다.

04 화면 상태 갱신

dashboard/detail 화면이 변경된 상태를 표시합니다.

YOLO 중간 결과와 최종 위험도 결과를 단계적으로 보여줄 수 있다는 점이 핵심입니다.



화면 갱신 구간: 연결 유지 → 결과 저장 → 이벤트 전달 → 화면 반영

서비스 처리 과정 요약

하나의 분석 작업은 입력 준비, AI 처리, 결과 저장, 화면 반영까지 같은 작업 단위로 추적됩니다.

01

입력 준비

분석 대상과 최신 센서값을 확인하고, AI 요청에 필요한 입력을 구성합니다.

02

AI 처리

AI 서버가 OpenCV 전처리, YOLO 분석, XGBoost 판단을 수행합니다.

03

결과 저장

백엔드가 콜백 결과를 저장하고, 분석 작업 상태와 배수구 상태를 갱신합니다.

04

화면 반영

WebSocket 이벤트로 dashboard/detail 화면에 변경된 상태를 반영합니다.

결과적으로 화면 응답, AI 처리, DB 저장, 화면 갱신을 분리하면서도 하나의 분석 작업으로 추적할 수 있습니다.

향후 RTSP, MQTT, 알림 확장

현재 구현은 sample image와 mock sensor data 기반이며, 실제 CCTV·센서·알림 자동화는 다음 확장 단계로 분리합니다.

현재 구현

sample / mock

sample image, mock sensor data, 비동기 분석, 결과 저장, 이벤트 기반 화면 반영을 시연합니다.

다음 확장

RTSP / MQTT

RTSP CCTV 입력과 MQTT sensor 연동을 실제 입력 채널로 검토합니다.

기대 효과

Alert / Report

상태 변화 알림과 리포트 자동화를 운영 확장 방향으로 제시합니다.

우선순위: 현재 흐름 안정화 -> 실제 입력 연동 -> 알림과 리포트 자동화.

시연 시나리오

발표자는 sample image와 mock sensor data가 들어와 작업 생성, 결과 저장, 화면 갱신으로 이어지는 장면을 보여줍니다.



입력 준비 화면: 시연용 sample image와 mock sensor data가 함께 들어간다고 설명

1

입력 준비

sample drain image와 mock sensor data를 준비합니다.

2

작업 생성

Backend가 AnalysisJob을 만들고 AI server 요청으로 이어집니다.

3

결과 저장

callback 결과가 PostgreSQL에 저장되고 job 상태가 갱신됩니다.

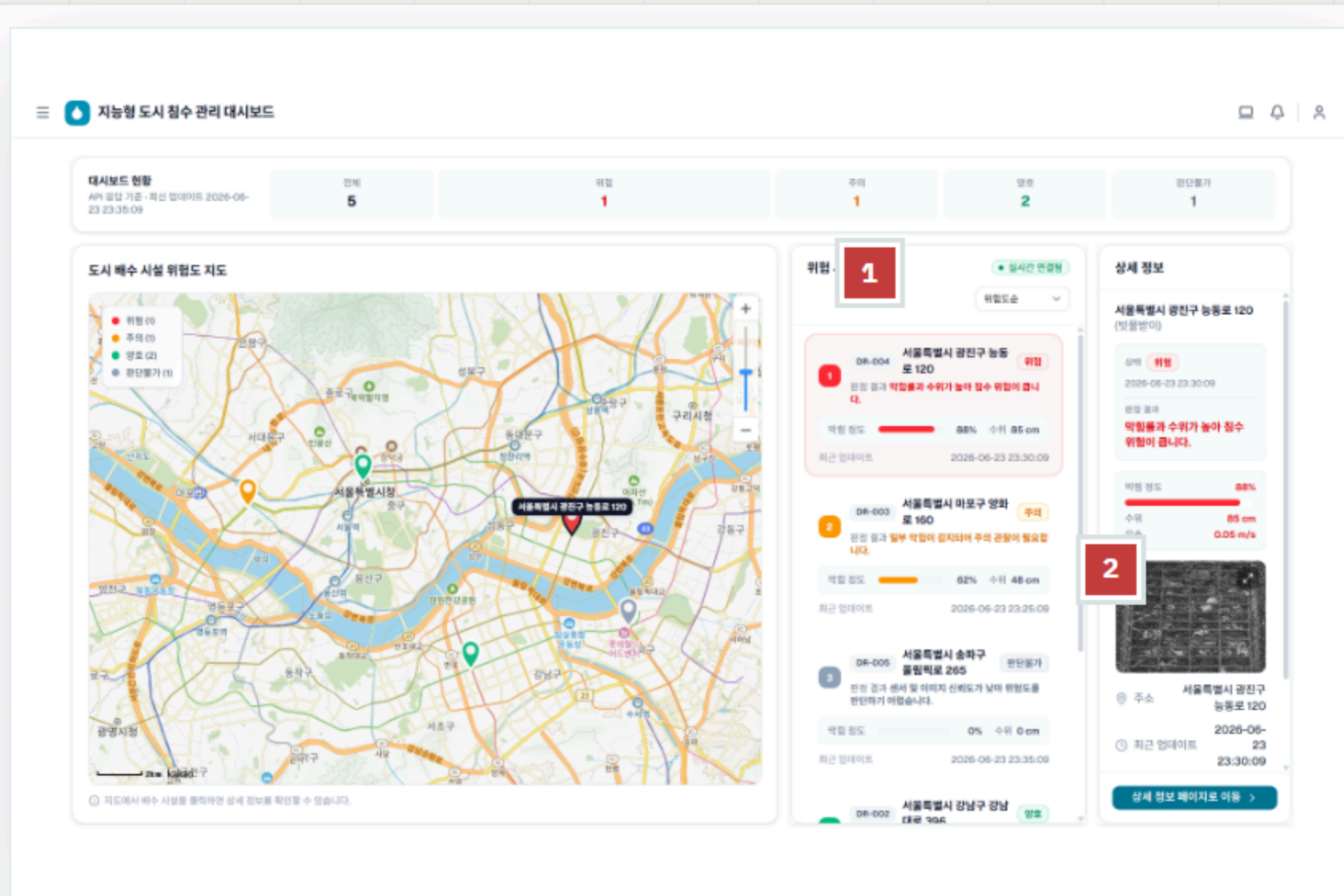
4

화면 갱신

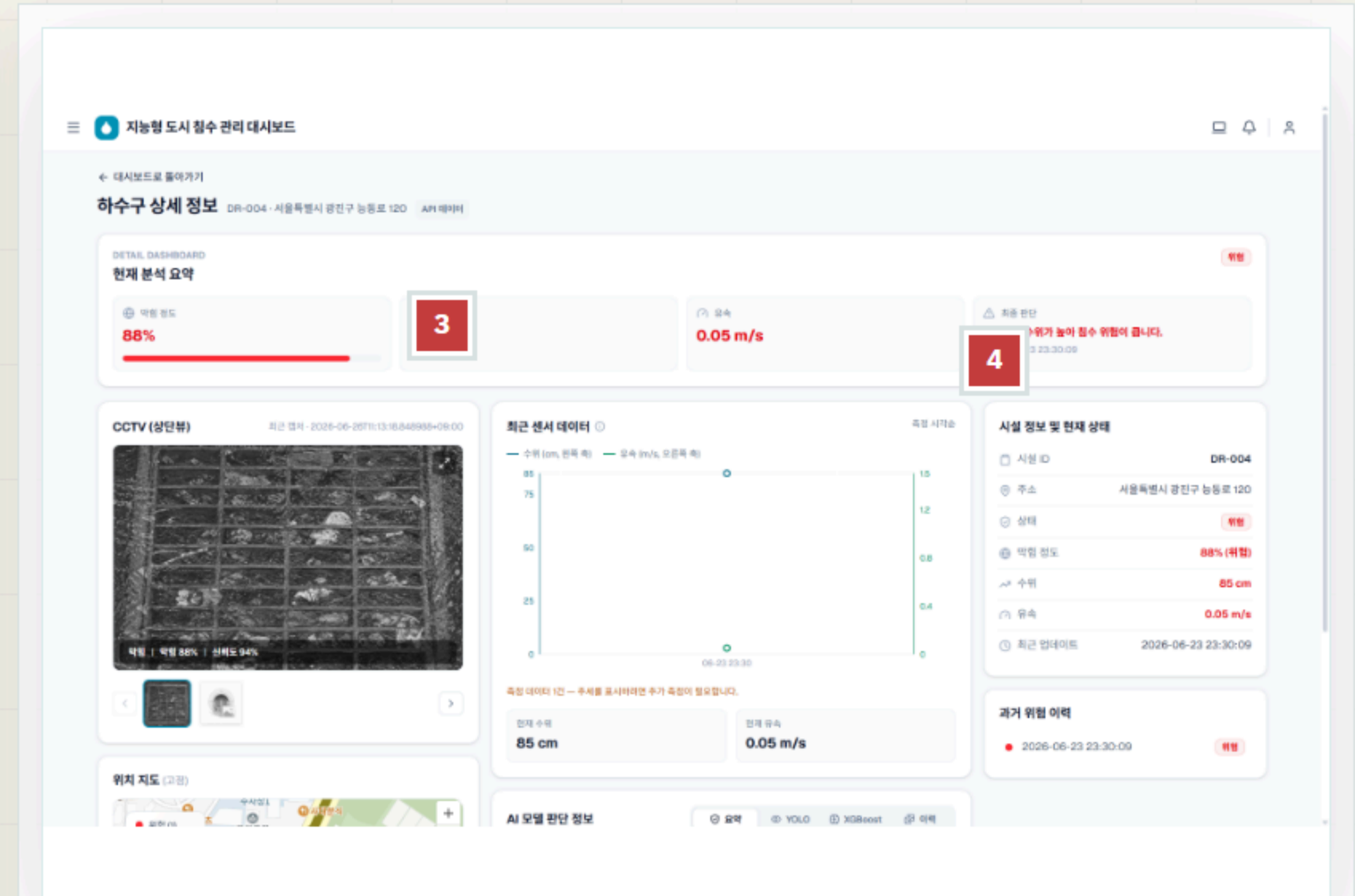
WebSocket event 이후 dashboard/detail UI가 최신 데이터를 반영합니다.

시연 흐름

대시보드에서 위험 시설을 고르고, 상세 화면에서 이미지·센서·AI 근거를 확인하는 순서로 시연합니다.



1. 위험 시설 확인 -> 2. 상세 화면 진입



3. 이미지·센서 근거 확인 -> 4. AI 결과와 이력 확인

Q & A

질문 받겠습니다

시설별 상태 확인, 위험 시설 우선 확인, 판단 근거 저장과
화면 반영 흐름을 제시했습니다.

정리

Q&A

기억할 핵심 1

현재 구현 범위는 sample images, mock sensor data, async analysis, result persistence, 이벤트 기반 화면 반영입니다.

기억할 핵심 2

RTSP, MQTT, 알림, 리포트는 현재 구현이 아니라 향후 확장 범위입니다.



우수주의보

GitHub Repository QR